

EE673 Assignment-2 : README

Overview

This submission documents the analysis and experiments performed for:

- Question 1 - TCP and UDP analysis using Wireshark
- Question 2 - TCP congestion control study using NetSim
- Question 3 - Bandwidth measurement using iPerf

No standalone executable is included. All results were produced by executing the listed networking tools and interpreting their outputs.

Software Prerequisites

Ensure the following tools are installed to reproduce the work:

- Wireshark (latest available version recommended)
 - NetSim (with Wireshark and WinPcap/Npcap support enabled)
 - iPerf3
 - Any modern web browser
 - LaTeX (optional; only required to rebuild the report)
-

Question 1 - Wireshark Analysis

This section examines TCP and UDP behavior through packet capture analysis.

Q1 (TCP) - Reproduction Steps

Step 1: Capture Network Traffic

1. Launch Wireshark.
2. Begin packet capture on the active network interface.
3. Using a browser, download the file:

<http://gaia.cs.umass.edu/wireshark-labs/alice.txt>

4. Open the upload page:

```
http://gaia.cs.umass.edu/wireshark-labs/TCP-wireshark-file1.html
```

5. Upload the previously downloaded file via HTTP POST.

6. Once the upload finishes and the success message appears, stop the capture.

Fallback: If live capture cannot be performed, open the supplied trace file `tcp-ethereal-trace-1`.

Step 2: Basic Filtering

Apply the following display filter to isolate TCP packets:

```
tcp
```

You should see the TCP three-way handshake, the HTTP POST transfer, and corresponding ACKs exchanged with `gaia.cs.umass.edu`.

Step 3: Targeted Filters

Use these filters for detailed inspection.

Client SYN

```
tcp.flags.syn == 1 && tcp.flags.ack == 0
```

Server SYN-ACK

```
tcp.flags.syn == 1 && tcp.flags.ack == 1
```

HTTP POST packet

```
http.request.method == "POST"
```

Retransmission detection

```
tcp.analysis.retransmission
```

Step 4: Measurements Extracted

From the TCP trace, the following metrics were determined:

- Client IP address and port
 - Server IP address and port
 - SYN segment sequence number
 - SYN-ACK sequence and acknowledgment numbers
 - Sequence number of the HTTP POST packet
 - Sequence numbers of the first six TCP data segments
 - Length of the first six segments
 - Round-Trip Time (RTT) for early packets
 - Estimated RTT using the TCP formula
 - Minimum advertised receiver window
 - Evidence of delayed ACKs
 - Presence or absence of retransmissions
 - Connection throughput
-

Step 5: Congestion Behavior (Q13–Q14)

To study slow start and congestion avoidance:

1. Open the TCP trace in Wireshark.
2. Select any client-to-server TCP segment.
3. Navigate to:

Statistics → TCP Stream Graph → Time-Sequence Graph (Stevens)

4. Examine the sequence-number vs time plot.
 5. Identify:
 - Start of slow start
 - End of slow start
 - Onset of congestion avoidance
-

Step 6: Throughput Check

For locally captured traces:

1. Record the total bytes transferred.
2. Record the total transfer duration.
3. Compute throughput as:

$\text{Throughput} = \text{Total Bytes} / \text{Total Time}$

Notes

- Relative TCP sequence numbering is enabled in Wireshark.
- TCP segment size is limited by the Ethernet MSS (1460 bytes).
- Small deviations from ideal behavior may arise due to delayed ACKs, RTT fluctuations, or implementation details.

Q1 (UDP) - Reproduction Steps

Step 1: Capture UDP Packets

Option A (live capture):

1. Start Wireshark.
2. Capture traffic for a short duration.
3. Typical DNS or SNMP activity will generate UDP packets.

Option B (recommended backup):

- Open the trace file `http-ethereal-trace-5`.

Step 2: Apply Filter

`udp`

Select any UDP packet and expand its header fields.

Step 3: Analysis Performed

From the chosen UDP packet:

- Identified all UDP header fields
- Determined the size of each field
- Interpreted the Length field
- Calculated the maximum UDP payload
- Found the largest source port value
- Verified the UDP protocol number in the IP header

- Compared request and reply port usage
-

Question 2 - TCP Congestion Control in NetSim

This experiment compares Old Tahoe, Reno, and Cubic TCP variants.

Network Topology

Client — Lossy Wired Link — Server

Simulation Configuration

Link Layer

- Bandwidth: 10 Mbps
- BER: $1e-7$
- Propagation delay: 25 ms

Transport Layer

- Protocol: TCP
- MSS: 1460
- Variants tested:
 - Old Tahoe
 - Reno
 - Cubic

Application

- FTP
- File size: $1e8$ bytes

Other Settings

- Simulation duration: 20 seconds
 - Wireshark capture: enabled
-

Execution Procedure

1. Launch NetSim.
2. Build the topology shown above.
3. Configure all parameters exactly as specified.
4. Run three simulations using:
 - Old Tahoe
 - Reno
 - Cubic
5. Open the Wireshark trace produced by NetSim.
6. Generate plots via:

Statistics → TCP Stream Graph → Time-Sequence / Window

Comparison Criteria

The following aspects were evaluated:

- Congestion window growth pattern
- Behavior under packet loss
- Recovery speed
- Average cwnd
- Link utilization

Question 3 - Bandwidth Measurement with iPerf

This experiment measures achievable throughput between two hosts.

Test Setup

- Two machines connected over an IP network
- One configured as server
- One configured as client

Execution Steps

Step 1: Launch Server

On machine A:

```
iperf3 -s -p <port_number>
```

Step 2: Forward Test (Client)

On machine B:

```
iperf3 -c <server_ip> -p <port_number>
```

Step 3: Reverse Test

```
iperf3 -c <server_ip> -p <port_number> -R
```

Step 4: Evidence Collection

Capture screenshots of:

- Server (forward)
 - Server (reverse)
 - Client (forward)
 - Client (reverse)
-

Final Notes

- Wireshark relative TCP numbering is enabled.
- No retransmissions were observed in the provided trace.
- All experiments follow the assignment guidelines.